Introduction to Object-Oriented Programming (OOP)

1. OOPs.

- Object Oriented Programming (OOP) is a programming approach that organizes code around objects. Objects contain data (characteristics) and methods (behaviour).
- OOP helps represent real world entities in code and makes large applications easier to organize and maintain.

2. Need of OOPs.

Procedural programming organizes code mainly around functions. As applications grow, managing related data and behaviour across many functions becomes difficult.

OOP helps by:

- Grouping related data and methods into classes.
- Controlling access to data.
- Reusing code.
- Organizing complex applications into manageable components.
- Making code easier to maintain and extend.

3. Class, Object, and Instance

- **Class:** A blueprint that defines the data and methods an object will have.
- **Object:** An entity created using a class.
- **Instance:** A specific object created from a class. The terms *object* and *instance* are commonly used interchangeably.

Example

```
class Car {
   String brand;
   void startEngine() {
      System.out.println("Engine started");
   }
}
```

Here, Car is a class with a brand field and a startEngine() method.

```
Car myCar = new Car();
myCar.brand = "Ford";
myCar.startEngine();
```

Here, myCar refers to an object of the Car class. Its brand is set to Ford, and its engine-starting method is called.

4. Key Benefits of OOP

- **Organization:** Keeps related data and behaviour together.
- **Data protection:** Controls access to internal data.
- **Code reuse:** Allows existing code to be reused.
- **Maintainability:** Makes code easier to understand, debug, and modify.
- **Extensibility:** Helps add new functionality with fewer unnecessary changes to existing code.